

Building AI Agents

with Spring AI

Timo Salm
Master Solutions Architect, Broadcom

September 2026

About Me

Timo Salm

Master Solutions Architect
VMware Tanzu by Broadcom



linkedin.com/in/timosalm



x.com/salmt0



github.com/timosalm



A Model Alone Is Not Enough

The Model

- Makes up facts
- Forgets past conversations
- Has outdated knowledge
- Does not know your data
- Cannot take action

Models are probabilistic, so the same prompt can lead to different answers.

The Usual Fixes Lead to

- Higher token costs
- New attack surfaces
- Governance challenges
- Little observability
- More code to maintain

Retrieval, memory and tools each solve one problem but add new ones.

The Answer

You need a harness around the model.

A harness connects the model to your data, memory and tools. It also keeps every call secure, governed and observable.

Spring AI helps you build this harness.

Spring AI Fundamentals

One consistent programming model for every AI provider, through two complementary APIs

ChatModel

The low level API behind every provider.

```
ChatResponse response = chatModel.call(  
    new Prompt("Tell me about Spring  
AI"));  
  
String text = response.getResult()  
    .getOutput().getText();
```

Use it when you need explicit control over every request.

ChatClient

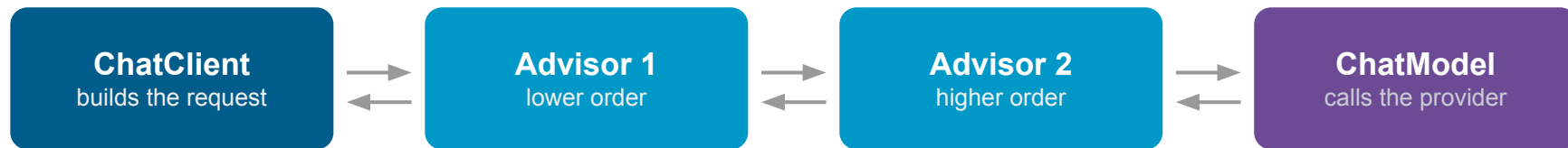
The recommended fluent API. It builds on ChatModel and adds useful defaults.

```
String answer = chatClient.prompt()  
    .user("Tell me about Spring AI")  
    .call()  
    .content();
```

Use it for everyday work in your applications.

The Advisors API

An advisor is an interceptor around every ChatClient call, with a before and an after step



Each advisor gets a ChatClientRequest on the way in and a ChatClientResponse on the way out.

```
class LoggingAdvisor implements CallAdvisor {
    public ChatClientResponse adviseCall(
        ChatClientRequest req, CallAdvisorChain chain) {
        log(req); // before
        var res = chain.nextCall(req); // rest of chain
        log(res); // after
        return res;
    }
    public String getName() { return "LoggingAdvisor"; }
    public int getOrder() { return LOWEST_PRECEDENCE; }
}
```

CallAdvisor and StreamAdvisor

Implement one or both to support the blocking call and the reactive stream.

Name and order

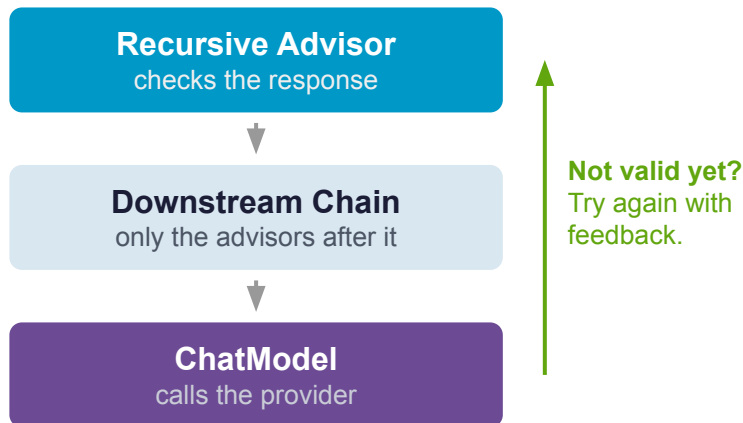
A lower order value runs first on the way in and last on the way out.

Shared context

A context map travels with the request, for example to pass a conversation id.

Recursive Advisors

Some advisors need to call the model more than once for a single user request



```
CallAdvisorChain downstream = chain.copy(this);
ChatClientResponse res = downstream.nextCall(req);
int attempt = 1;
while (!matchesSchema(res) && attempt < max) {
    req = addFeedback(req, res);
    res = downstream.nextCall(req);
    attempt++;
}
return res;
```

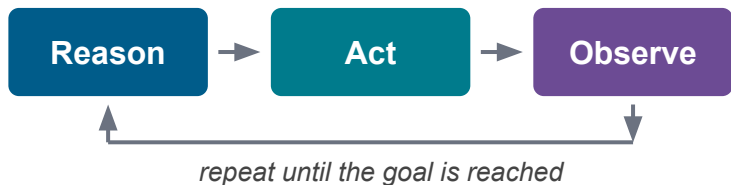
chain.copy(this) makes sure the advisors before it do not run again.

Examples: ToolCallingAdvisor, StructuredOutputValidationAdvisor

What Is Agentic AI?

From answering single questions to reaching goals on its own

An AI agent uses a model to **decide its next step**, **acts through tools**, and repeats this **loop** until the goal is reached.



Agentic systems come in two forms. A workflow follows a path you define in code, while an agent chooses its own path.

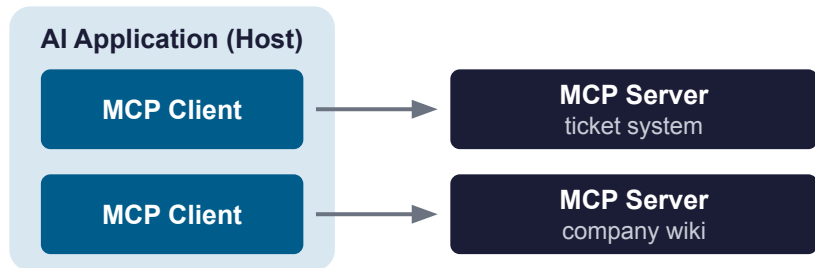
THE EVOLUTION OF LLM BASED SOLUTIONS

- **Standalone LLMs**
Prompts in, text out for language tasks
- **Grounded Generation (RAG)**
Answers based on your own business data
- **Tool Calling and MCP**
The model can trigger actions in real systems
- **Agentic Systems**
Workflows and agents built on models and tools

Excursion: Model Context Protocol (MCP)

An open standard that connects AI applications to tools and data, like USB-C for AI

What MCP Is



Three primitives

Tools to act, resources for context, and reusable prompts.

Two transports

stdio locally, Streamable HTTP over the network.

Open and vendor neutral

Governed by the Agentic AI Foundation, not one vendor.

MCP in Spring AI

As a client `spring-ai-starter-mcp-client`

```
@Bean
ChatClient chatClient(ChatClient.Builder cb,
    ToolCallbackProvider mcpTools) {
    return cb.defaultTools(mcpTools).build();
}
```

As a server `spring-ai-starter-mcp-server-webmvc`

```
@McpTool(description = "Get project releases")
List<Release> releases(
    @McpToolParam(description = "Project slug")
    String slug) { ... }
```

Remote MCP tools behave like your own @Tool methods. The same tool calling loop runs, and your prompting code stays the same.

Two Approaches to Orchestrate LLMs and Tools

Workflows

Your code defines the path, and the model does its work inside each step.

- ✓ Predictable and consistent results
- ✓ Easier to test and debug
- ✓ Lower cost and latency
- ✓ Ideal for well defined, repeatable tasks
- ✗ Cannot adapt to unexpected subtasks
- ✗ You must split the task up front

Agents

The model decides which steps to take and which tools to use.

- ✓ Flexible for open ended problems
- ✓ Handles tasks whose steps are unknown up front
- ✓ Scales well in trusted environments
- ✗ Higher cost and latency
- ✗ Errors can add up over many steps
- ✗ Harder to debug and control
- ✗ Needs thorough testing and guardrails

Start with a workflow for predictability. Use an agent only when you truly need its flexibility.

Workflow Patterns

Your code decides the steps. Pick the simplest pattern that fits the task

Chain

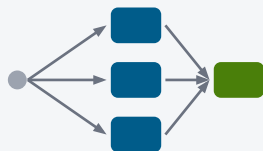


Splits a task into steps. Each output feeds the next model call.

Sample code

`spring-ai-examples`

Parallelization

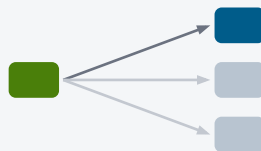


Runs independent calls at the same time and combines the results.

Sample code

`spring-ai-examples`

Routing

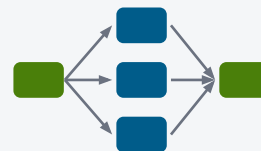


Classifies the input and sends it to a specialized prompt or model.

Sample code

`spring-ai-examples`

Orchestrator-Workers



Splits the task at runtime and lets workers handle the parts.

Sample code

`spring-ai-examples`

Evaluator-Optimizer

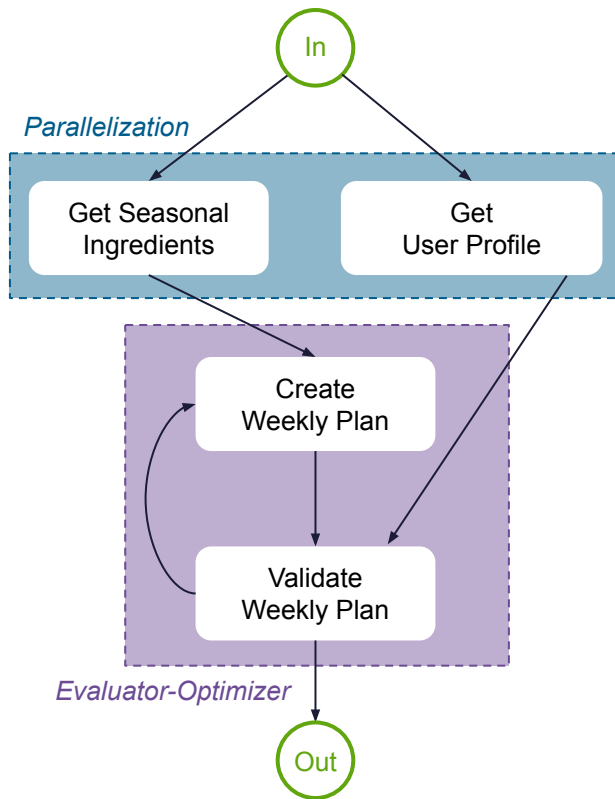


Generates an answer, evaluates it, and retries with the feedback.

Experimental

`SelfRefineEvaluationAdvisor`

Sample Application



Days & Meals

Monday All ☒ Breakfast ☒ Lunch ☒ Dinner	Tuesday All ☒ Breakfast ☐ Lunch ☐ Dinner	Wednesday All ☒ Breakfast ☒ Lunch ☒ Dinner	Thursday All ☒ Breakfast ☐ Lunch ☐ Dinner	Friday All ☐ Breakfast ☒ Lunch ☐ Dinner	Saturday All ☒ Breakfast ☐ Lunch ☐ Dinner	Sunday None ☒ Breakfast ☒ Lunch ☒ Dinner
---	--	--	---	---	---	--

Country (ISO code)

DE

Additional instructions

Prefer quick recipes with preparation time under 30 minutes

Generate Plan

Your Weekly Plan

MONDAY

BREAKFAST

Wild Garlic Spinach Omelette 10 min

Ingredients

- 3 pieces Eggs
- 60 g Spinach
- 1/2 bunch Wild garlic
- 1 tbsp Chives
- 1 tsp Olive oil
- to taste Salt
- to taste Pepper

Instructions

Beat eggs with salt and pepper. Sauté wild garlic in oil, add spinach until just wilted. Pour in eggs and cook gently, folding until set. Finish with chopped chives.

NUTRITION

Calories	270 kcal
Protein	18,0 g
Carbs	4,0 g
Fat	19,0 g
Sodium	280 mg

LUNCH

Patterns for a Single Agent

Components in the harness that keep one agent focused, on track, and under control

Agent Skills

Reusable instruction packages. Only the name and description are loaded at the start, and the full instructions follow when a task needs them.

Experimental

SkillsTool

Tool Search

The agent gets one search tool and loads only the few tool definitions it needs for the current step.

GA

ToolSearchToolCallingAdvisor

Plan and Execute

The agent writes a task list first and works through it step by step, so it does not lose track.

Experimental

TodoWriteTool

Human in the Loop

The agent asks when the input is unclear and asks for approval before risky actions.

Experimental

AskUserQuestionTool

Tool Search is part of Spring AI since 2.0. The other tools come from the community project spring-ai-agent-utils.

Multi-Agent Patterns

Keep one agent in charge, and add a second one only where it pays off

LLM as a Judge

A second agent rates the answer and sends feedback until it passes. A different judge model avoids bias.

Experimental

`SelfRefineEvaluationAdvisor`

Subagents

The main agent delegates a task to an agent with its own context, tools, and model, and reads only the result.

Experimental

`TaskTool`

Agent-to-Agent (A2A)

An open protocol that lets agents in different systems find and call each other, like MCP does for tools.

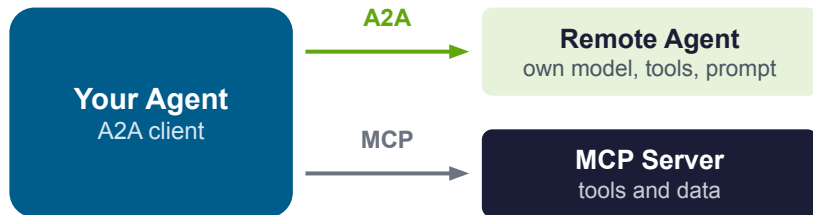
Experimental

`spring-ai-a2a`

Excursion to the Agent-to-Agent Protocol (A2A)

MCP connects an agent to tools. A2A connects an agent to other agents

What A2A Is



Agent Card

A JSON description of the agent and its skills, published at a well known URL.

Messages and tasks

Agents talk over JSON-RPC and track longer work as tasks.

Open and vendor neutral

Started by Google and now hosted by the Linux Foundation.

A2A in Spring AI

Experimental

As a server `spring-ai-a2a-server-autoconfigure`

- 1 **Your capabilities** Still defined as usual, with `@Tool` methods, advisors, and prompts on your `ChatClient`.
- 2 **An `AgentCard` bean** Describes the name, description, and skills of your agent for discovery.
- 3 **An `AgentExecutor` bean** Hands incoming messages to your `ChatClient`, usually via `DefaultAgentExecutor`.

Spring AI publishes the card and handles the protocol for you.

As a client `a2a-java-sdk-client`

Wrap the call to a remote agent in a `@Tool` method. The model then decides on its own when to delegate.

Learn Spring AI

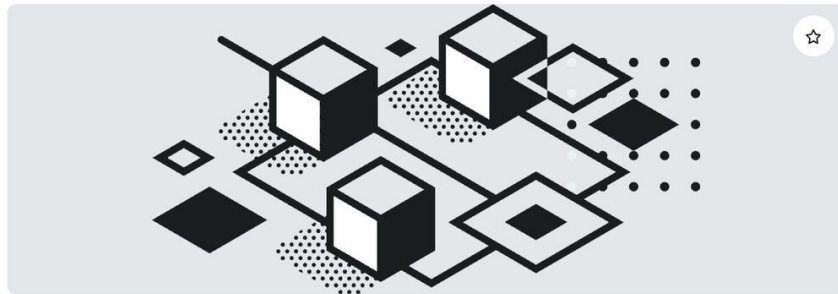
With my free course on Spring Academy

Build AI applications and agents
step by step, with **hands-on labs**.

Free of charge. You only need to register.



spring.academy/courses/spring-ai-intro



Beginner



Spring AI Introduction

This course introduces you to Spring AI, a framework that enables you to integrate GenAI capabilities into your enterprise applications.

You'll learn the fundamentals of building AI-powered Spring Boot applications, from chat interactions to Retrieval-Augmented Generation (RAG), tool calling, testing, observability, and agentic AI with the Model Context Protocol (MCP).

Each section is followed by a hands-on lab in which you apply what you've just learned and, step by step, build a complete AI-powered support assistant.

24 Lessons

7h 33m

Start Course

Thank You



github.com/timosalm/spring-ai-nutrition-planner